



МИНОБРНАУКИ РОССИИ

Федеральное государственное бюджетное образовательное учреждение
высшего образования

«МИРЭА – Российский технологический университет»

РТУ МИРЭА

Институт Информационных технологий

Кафедра Математического обеспечения и стандартизации информационных
технологий

Отчет по Модулю 6 –Сетевое взаимодействие

по дисциплине «Разработка клиент-серверных мобильных приложений»

Выполнили:

Студенты группы ИКБО-65-23

Олефиоров Г.Г.

Проверил:

Дворникова Е.М.

2026 г.

Задание 1: Приложение “Фотокаталог”

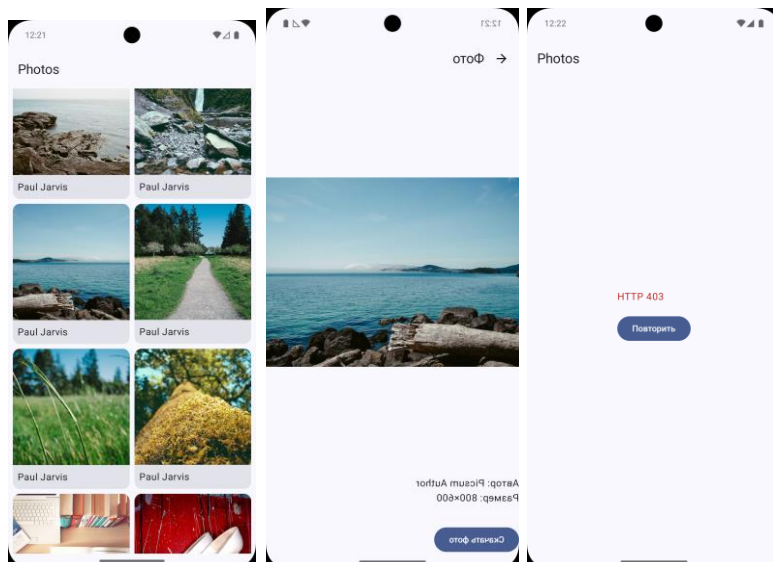
Функционал приложения:

Создать приложение на **Retrofit + Clean Architecture**, которое:

- показывает список случайных красивых фотографий;
- при клике на фото открывает экран детализации с большой картинкой;
- позволяет скачать фото в папку Downloads через SAF;
- API: <https://picsum.photos/v2/list> (открытый, без регистрации).

Требования к приложению:

1. Экран списка фотографий:
 - Grid или LazyVerticalGrid (по 2 столбца);
 - Для каждой фотографии: маленькая превьюшка (thumbnailUrl), автор (author), размеры (width × height);
 - При клике на фото — переход на экран детализации.
2. Экран детализации:
 - Большая картинка (download_url или url) через Coil/Glide;
 - Информация: автор, размеры, ссылка;
 - Кнопка «Скачать фото» - через SAF сохраняем в папку Downloads.
3. Clear Architecture;
4. Обработка состояний: Loading (CircularProgressIndicator); Success (список); Error (текст ошибки + кнопка «Повторить»).



Моя реализация:

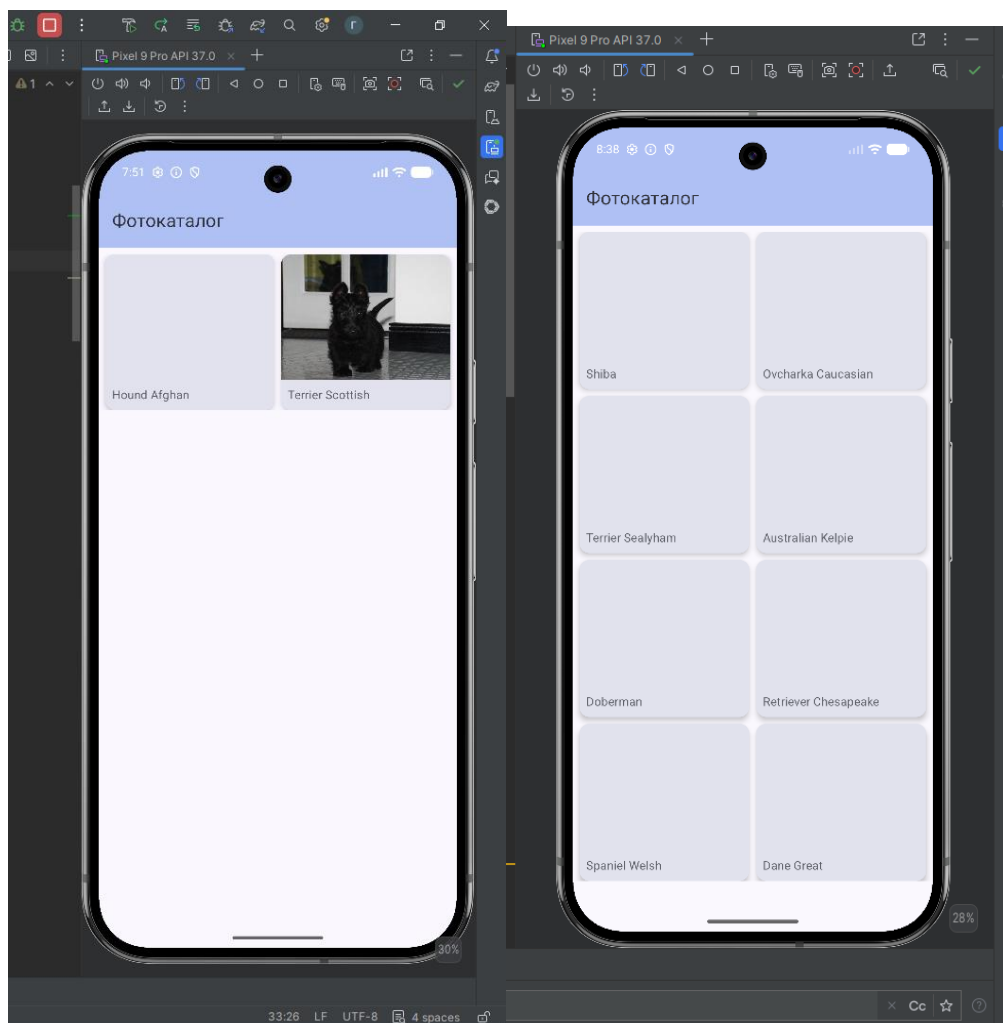


Рисунок 1 – Экран списка фотографий

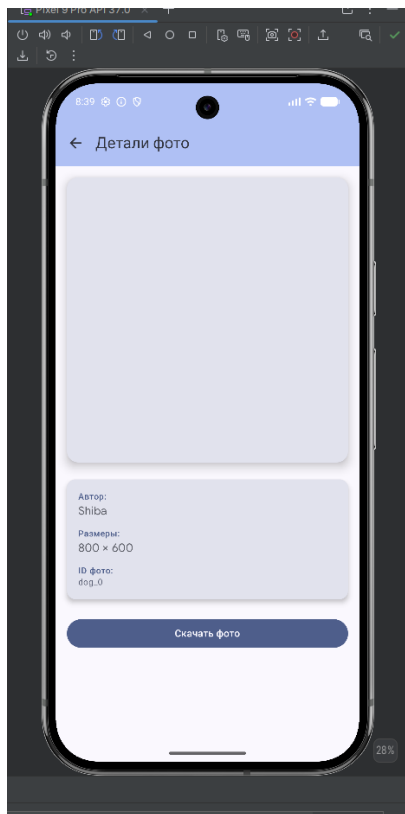


Рисунок 2 – Экран детализации

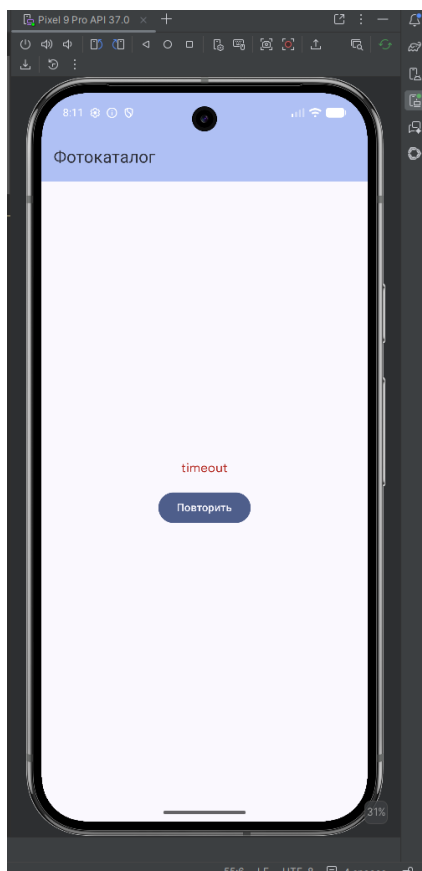


Рисунок 3 – Экран при ошибке

Ссылка на гит:

<https://github.com/oggiorgi/MIREA/tree/main/6sem/kotlin/6module/app/code1>

Задание 2: Приложение “Нобелевские лауреаты”

Функционал приложения:

Создать приложение на **Ktor Client + Clean Architecture**, которое:

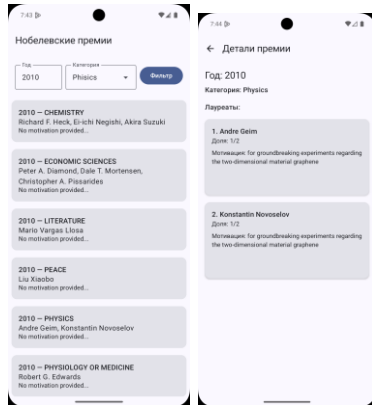
- показывает список лауреатов Нобелевской премии;
- позволяет фильтровать по году и/или категории;
- при клике на лауреата открывает экран с подробной информацией;
- API: <https://api.nobelprize.org/2.1/nobelPrizes?limit=25&offset=0> (можно добавить параметры: nobelPrizeYear=2023, nobelPrizeCategory=physics и т.д.)

Требования к приложению:

1. Экран списка лауреатов (LazyColumn или LazyVerticalGrid)
 - Для каждого элемента:
 - Год премии;
 - Категория (physics, chemistry, literature, peace, medicine, economics);
 - Имя лауреата (fullName);
 - Краткое описание (motivation) — первые 100 символов.
 - Поля для фильтра: год (например, от 1901 до текущего) и категория (выпадающий список).
2. Экран детализации лауреатов:
 - Полное имя;
 - Год и категория;
 - Полное описание (motivation);
 - Страна (birthCountry / birthPlace);
 - Фото (если есть в ответе — поле portraitUrl или аналогичное).

3. Clean Architecture;

4. Обработка состояний: Loading (CircularProgressIndicator); Success (список); Error (текст ошибки + кнопка «Повторить»).



Моя реализация:

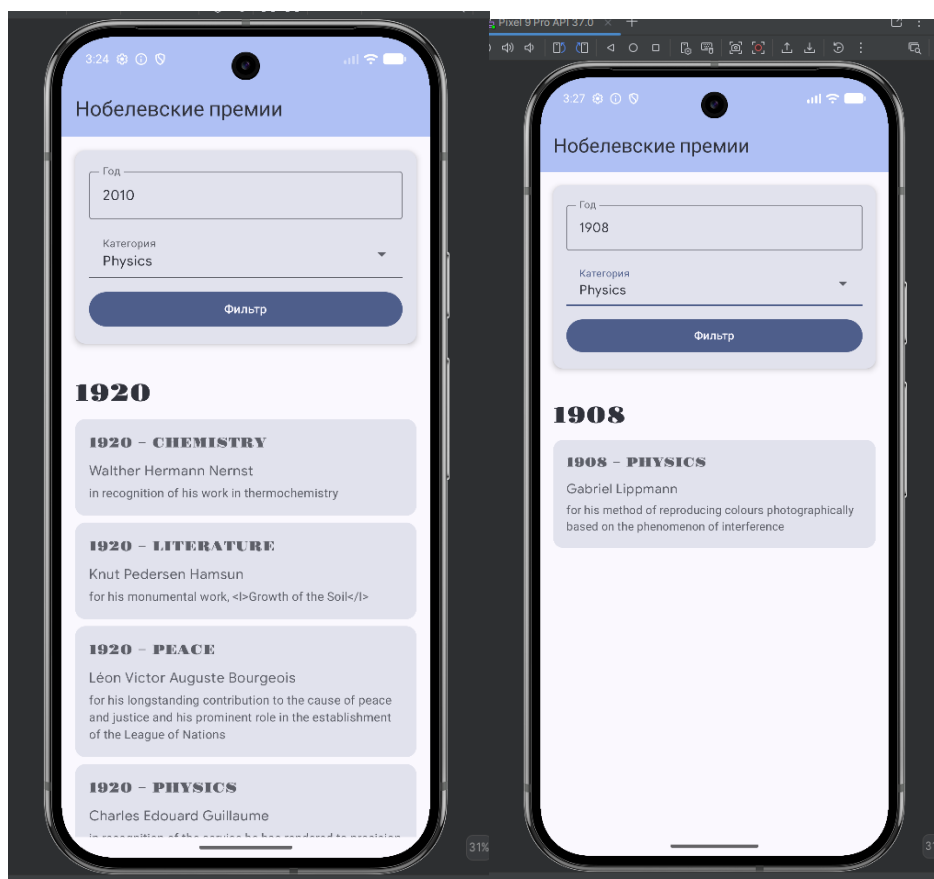


Рисунок 4 - Экран списка лауреатов



Рисунок 5 - Экран детализации лауреатов

Ссылка на гит:

<https://github.com/oggiorgi/MIREA/tree/main/6sem/kotlin/6module/app/code2>

Задание 3: Приложение “Авторизация”

Функционал приложения:

Создать приложение на **Ktor Client (или Retrofit) + Clean Architecture**, которое:

- позволяет пользователю авторизоваться (логин + пароль);
- после успешного входа показывает список пользователей;
- при клике на пользователя открывает экран с его детальной информацией;
- сохраняет токен авторизации в DataStore;
- обрабатывает выход из аккаунта (удаление токена).
- API: <https://dummyjson.com/> (открытый тестовый API)

Обязательные эндпоинты:

- POST /auth/login — авторизация (возвращает token и данные пользователя);
- GET /users — список всех пользователей (требуется Bearer Token);
- GET /users/{id} — детальная информация о пользователе (требуется Bearer Token);

Тестовые учётные данные:

- username: emilys
- password: emilypass

Требования к приложению:

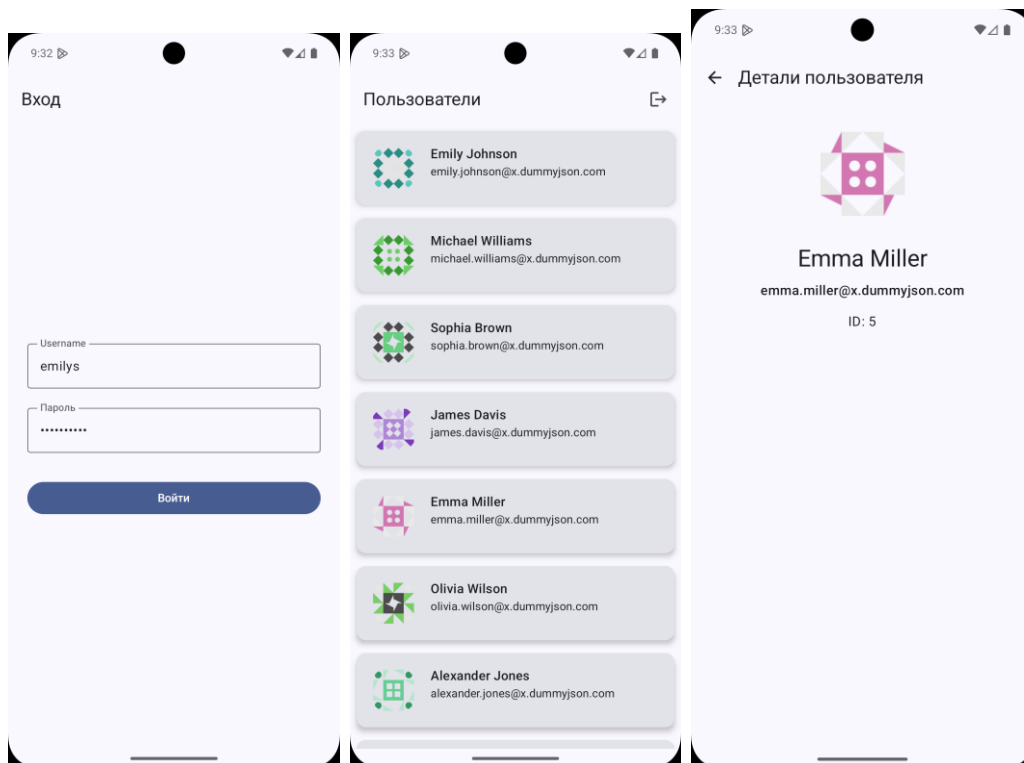
1. Экран авторизации (LoginScreen):
 - поля для username и password;
 - кнопка «Войти»;
 - индикатор загрузки;
 - обработка ошибок («Неверные данные», «Нет соединения» и т.д.);
 - после успеха — переход на список пользователей.
2. Экран списка пользователей (UsersListScreen):
 - LazyColumn / LazyVerticalGrid;
 - для каждого пользователя: аватар (image), имя (firstName + lastName), username, email;
 - при клике — переход на детали.
3. Экран детальной информации о пользователе (UserDetailScreen):
 - большая аватарка;
 - имя, фамилия и email
 - кнопка «Выйти» (очистка токена + возврат на логин)
4. Clean Architecture;

5. Хранение токена:

- использовать DataStore (Preferences DataStore);
- сохранять токен после успешного логина;
- добавлять Authorization: Bearer \$token ко всем запросам после авторизации;
- при выходе — очищать токен.

6. Обязательная обработка состояний:

- Loading (CircularProgressIndicator)
- Success (данные)
- Error (текст ошибки + кнопка «Повторить»)



Моя реализация:

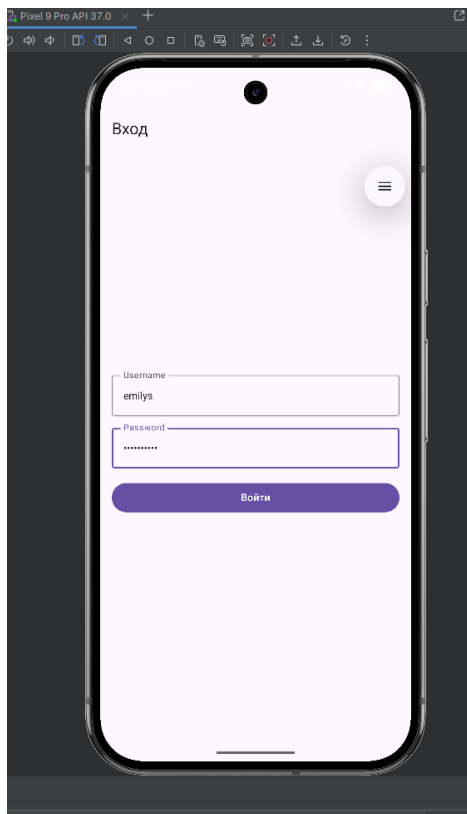


Рисунок 6 - Экран авторизации

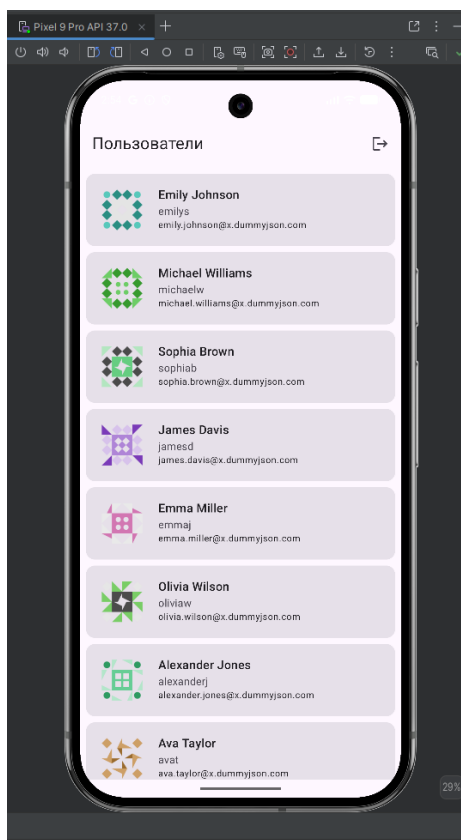


Рисунок 7 - Экран списка пользователей

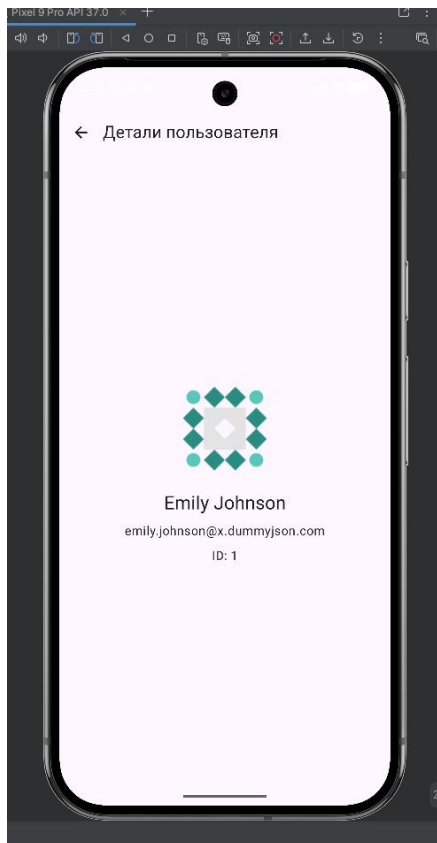


Рисунок 8 - Экран детальной информации о пользователе

Ссылка на гит:

<https://github.com/oggiorgi/MIREA/tree/main/6sem/kotlin/6module/app/code3>

Задание 4: Серверное приложение “Nobel Prize API”

Создать сервер API на фреймворке Ktor, который будет имитировать часть функционала Nobel Prize API + иметь авторизацию.

Требования:

1. Эндпоинты:

- POST /auth/login - логин/пароль - JWT-токен;
- GET /prizes - список всех премий (защищённый);
- GET /prizes/{year}/{category} - детальная премия (защищённый);
- GET /prizes/{year}/{category}/laureates - список лауреатов премии (защищённый)

2. **Авторизация:** JWT, срок 30 минут, ключ подписи — любой ≥ 32 символа;
3. **Данные:** в памяти (List<NobelPrize>) или в файлах;
4. Логирование запросов в консоль – CallLogging;
5. **Clean Architecture;**
6. **Все данные для заполнения структур можно посмотреть здесь:**
<https://www.nobelprize.org/about/developer-zone-2/>

Моя реализация

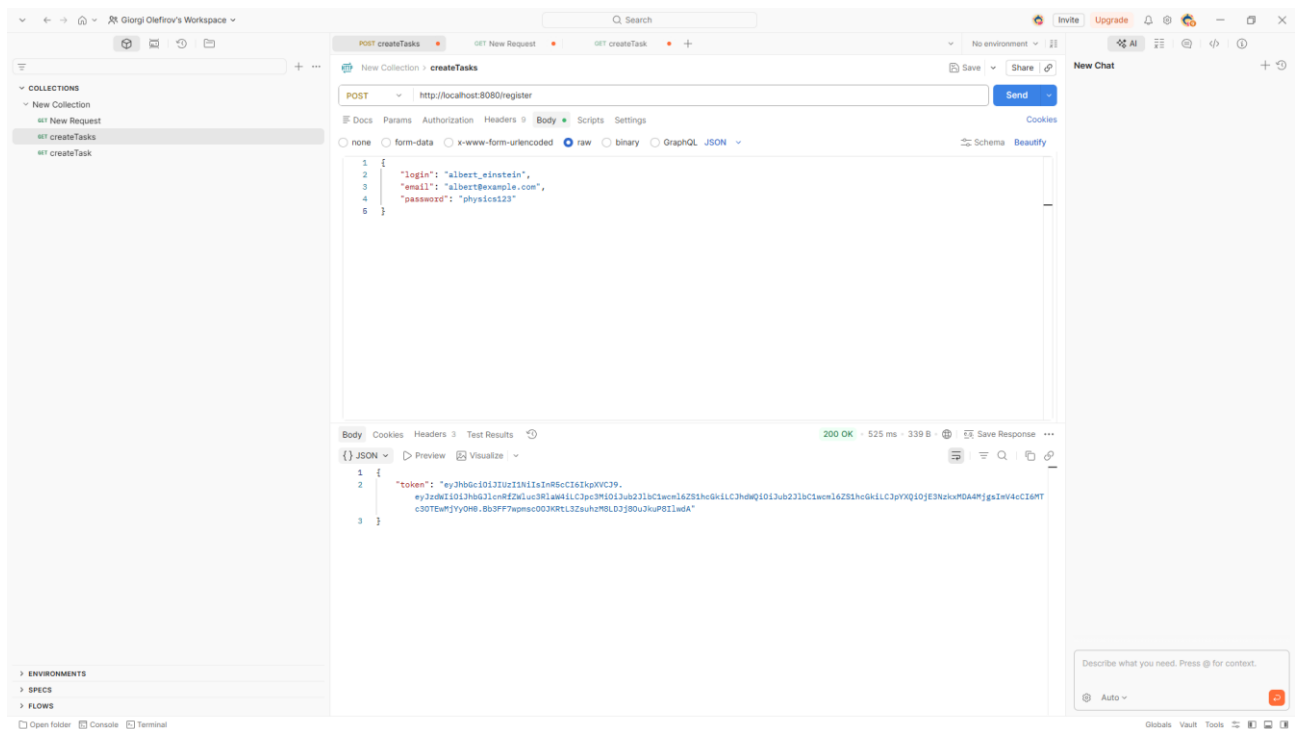


Рисунок 9 - Регистрация нового пользователя



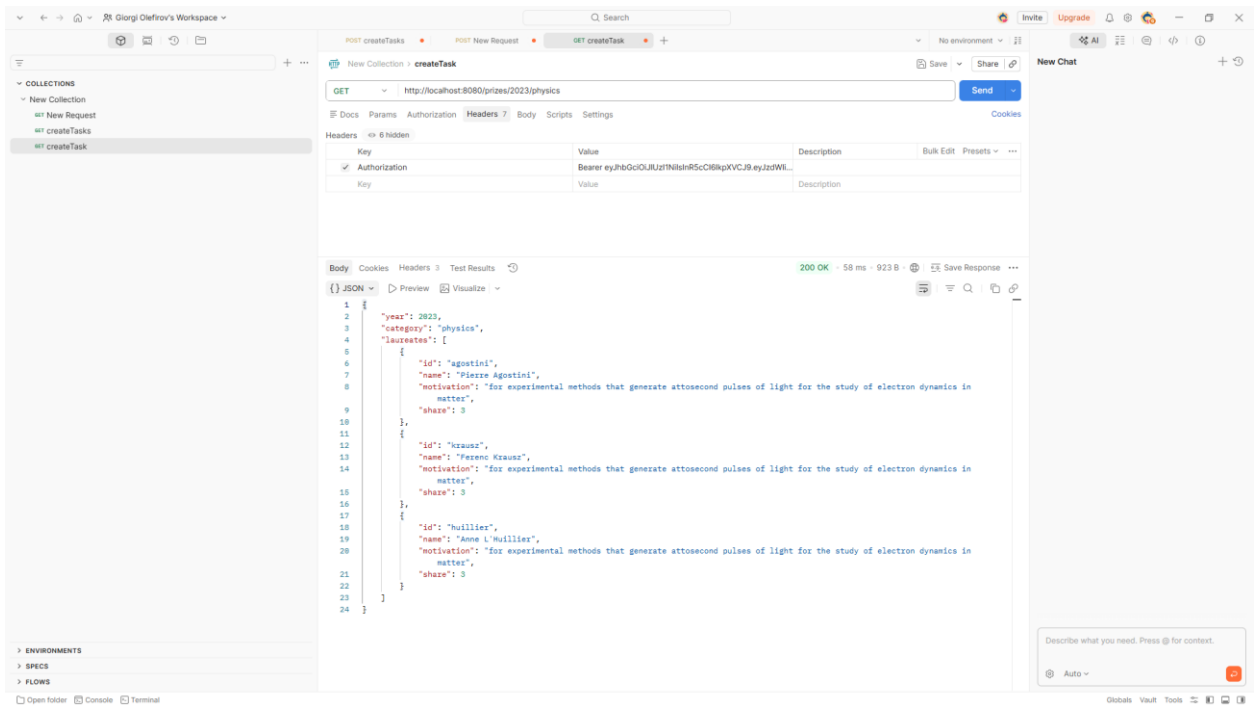


Рисунок 12 – Детали конкретной премии(физика)

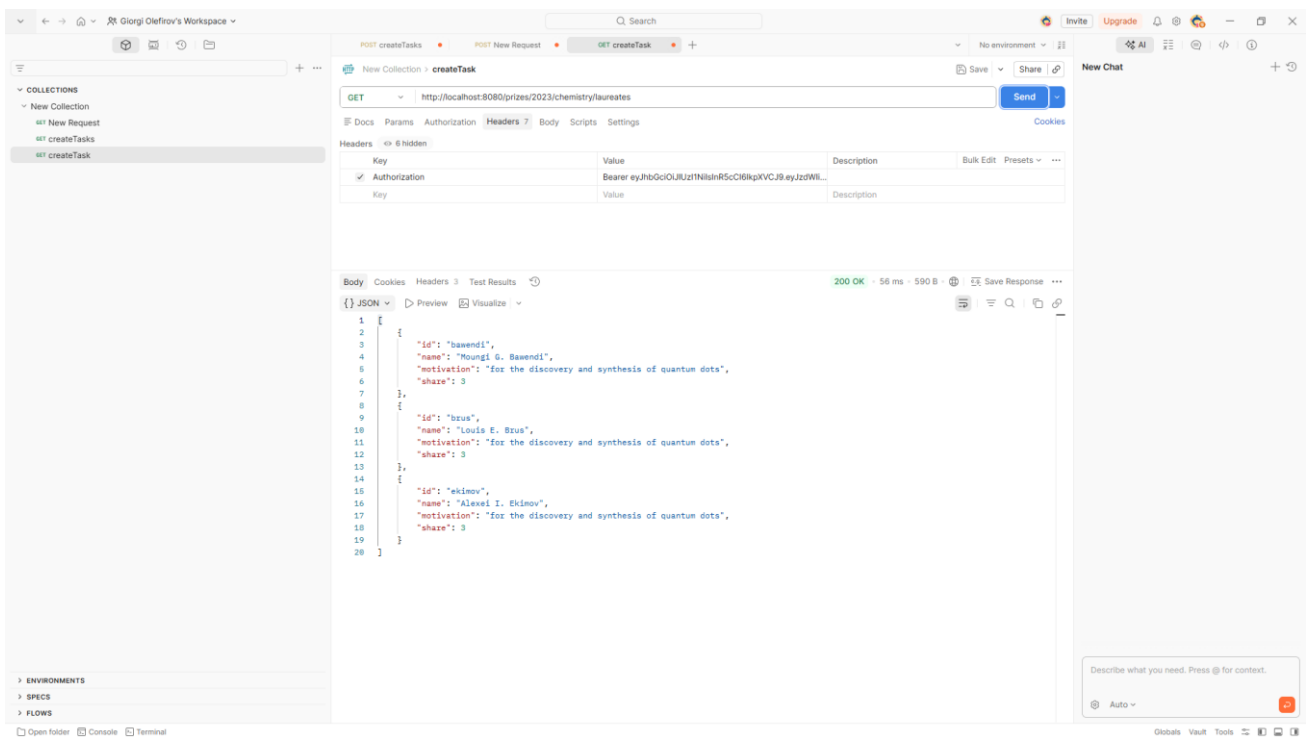


Рисунок 13 - Список лауреатов премии

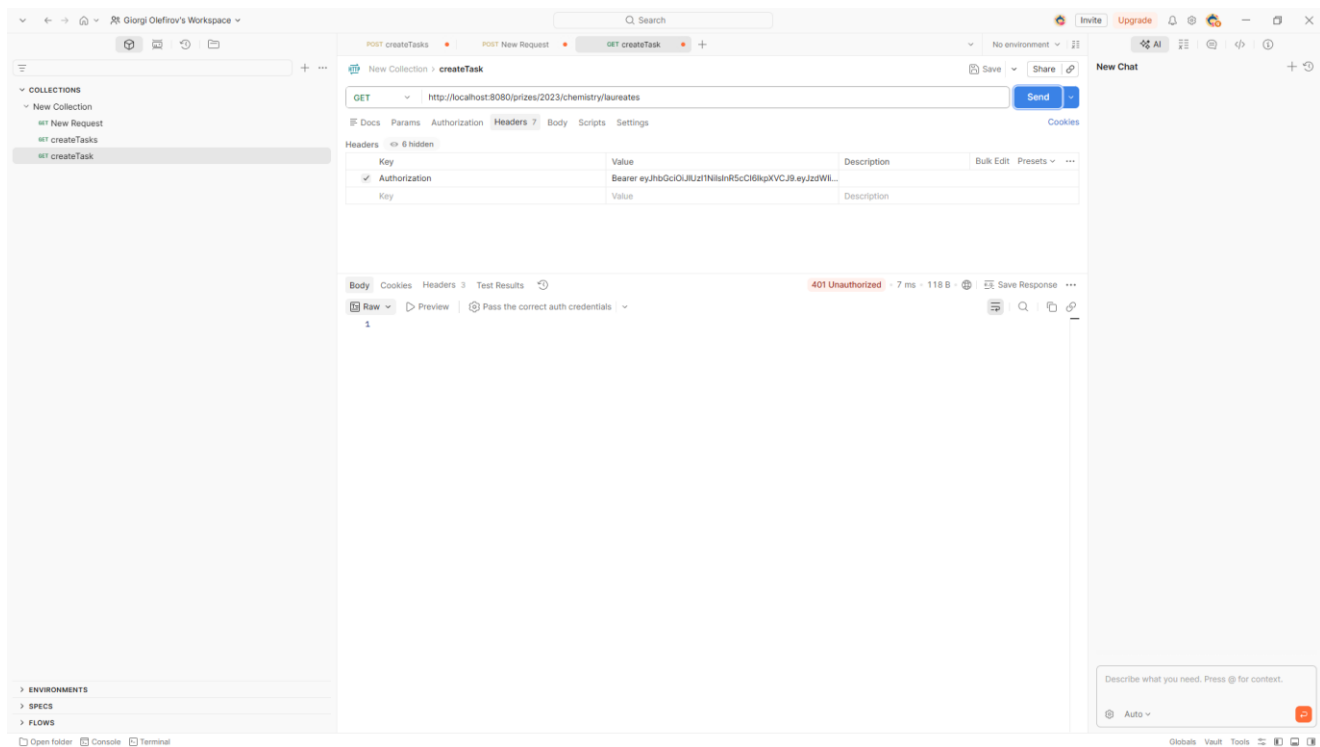


Рисунок 14 - Доступ с неправильным токеном

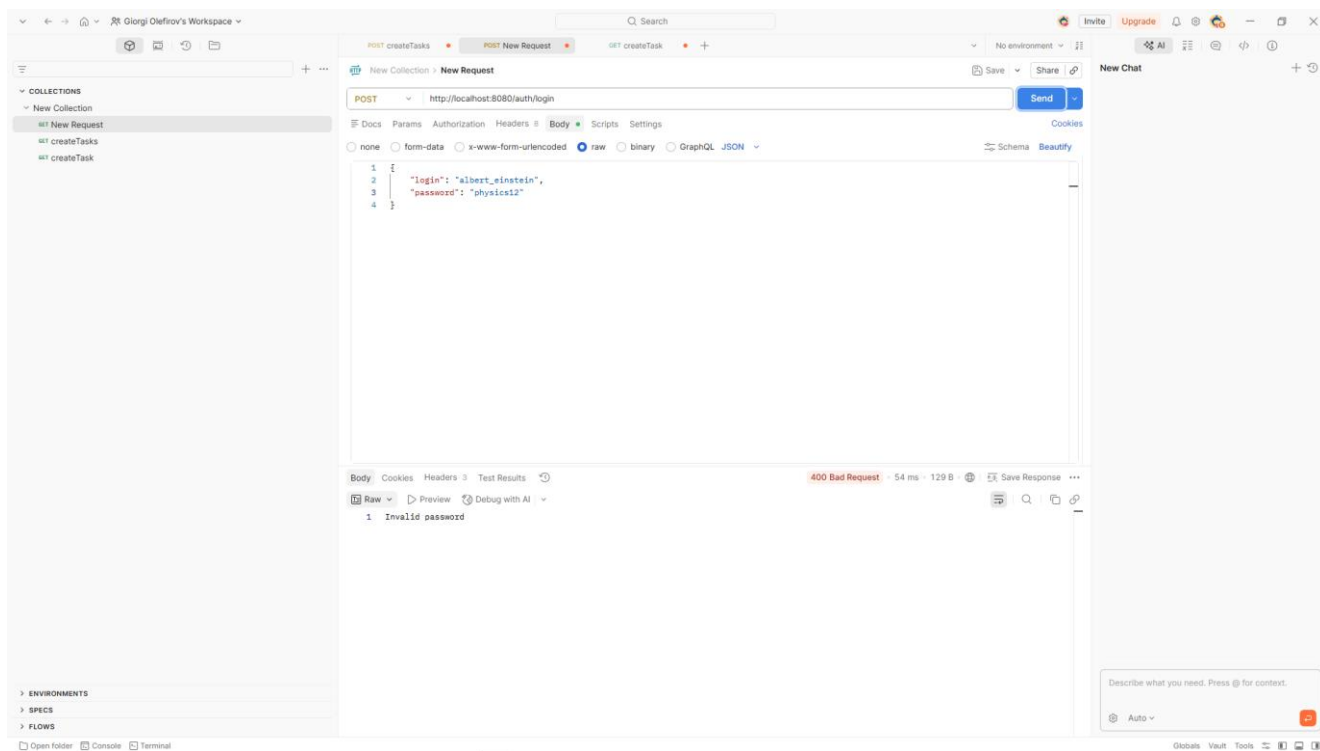


Рисунок 15 - Неверный пароль

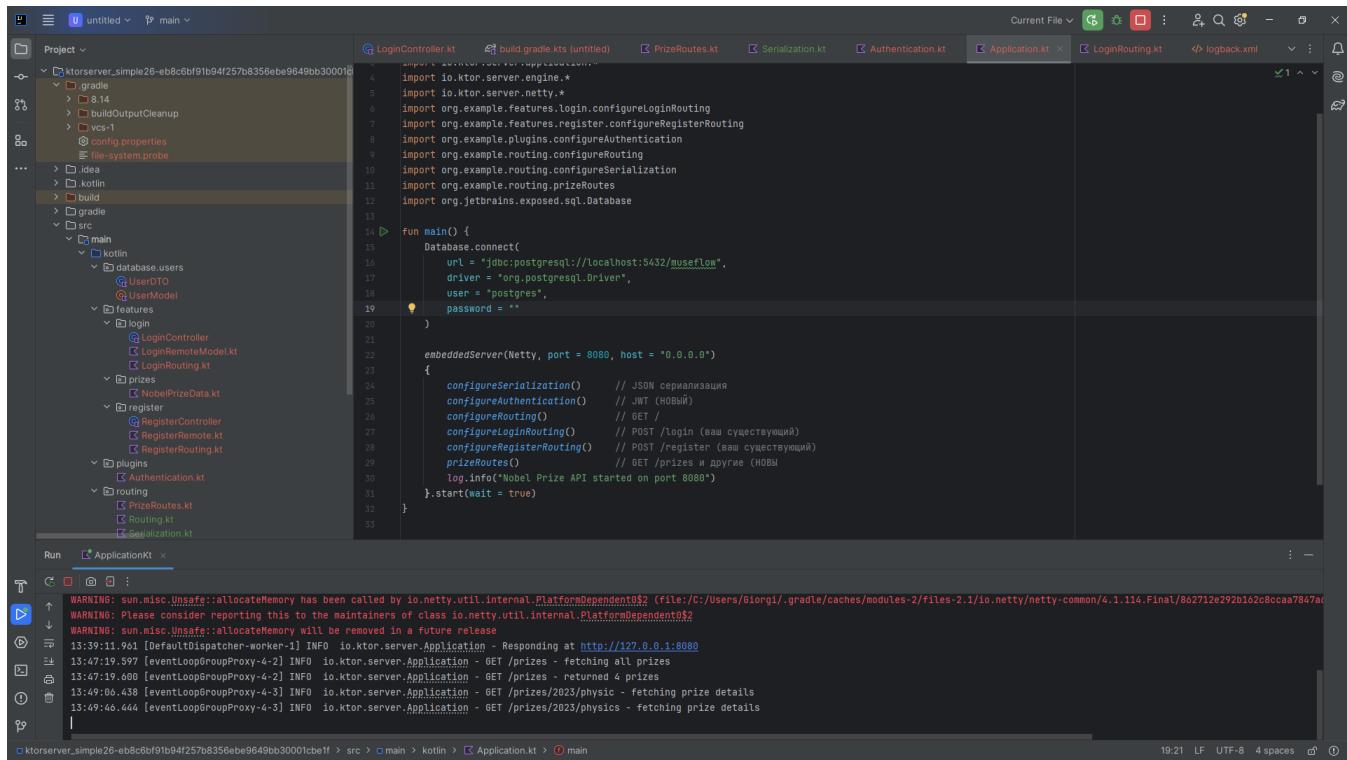


Рисунок 16 – Запущенный сервер с логами

Ссылка на гит:

<https://github.com/oggiorgi/MIREA/tree/main/6sem/kotlin/6module/app/4code>

Задание 5: Серверное приложение “Nobel Prize API” с хранением данных в neon.tech

Доработать сервер API из задания 4, который будет использовать для хранения данных удаленную СУБД PostgreSQL (сервис <https://neon.tech>)

Требования:

- Авторизует пользователей;
- Получает данные из публичного Nobel Prize API (допускается разовый ручной перенос);
- Сохраняет их в PostgreSQL;
- Позволяет пользователям добавлять/удалять любимые премии;
- Логирование запросов в консоль – CallLogging;
- Clean Architecture;

- Добавить описание сервиса с помощью SwaggerUI и/или Redocly:
<https://smiley4.github.io/ktor-openapi-tools/latest/>

Все данные для заполнения структур можно посмотреть здесь:
<https://www.nobelprize.org/about/developer-zone-2/>

Рекомендуемые таблицы:

1. users — id, username, password_hash, role;
2. prizes — id, award_year, category, full_name, motivation, detail_link;
3. laureates — id, prize_id, full_name, portion, motivation, portrait_url;
4. user_prizes — user_id, prize_id, added_at.

Рекомендуемые эндпоинты:

Открытые:

- POST /login — авторизация;
- GET /prizes — список премий (кэш в БД + обновление из API);

Защищённые:

- GET /users/me — профиль;
- GET /users/me/prizes — избранные премии;
- POST /users/me/prizes/{prizeId} — добавить премию;
- DELETE /users/me/prizes/{prizeId} — удалить премию.

Моя реализация:

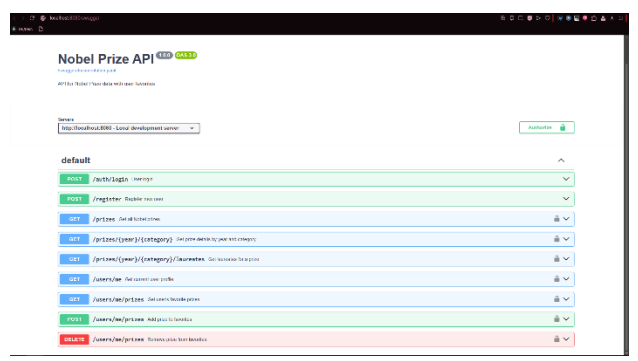
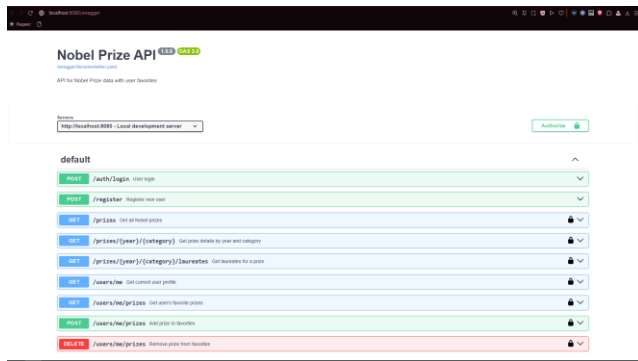


Рисунок 17 - SwaggerUI

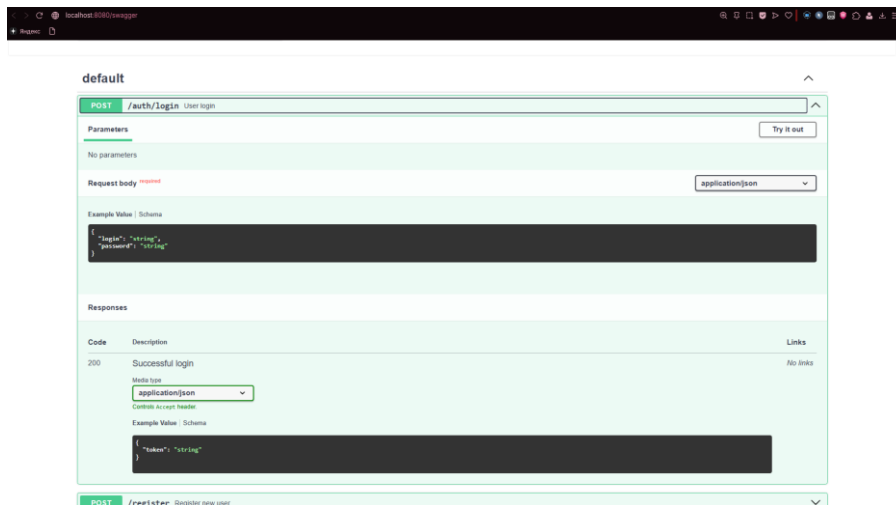


Рисунок 18 – SwaggerUI

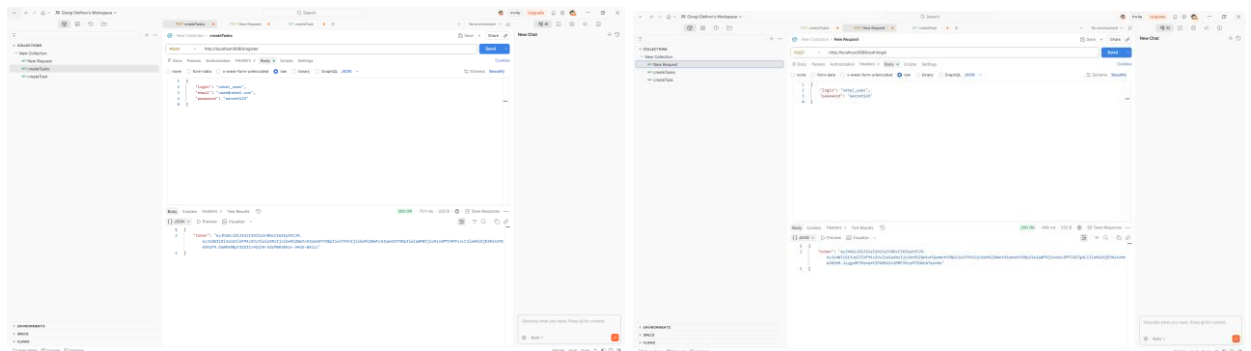


Рисунок 19 – Регистрация и логин

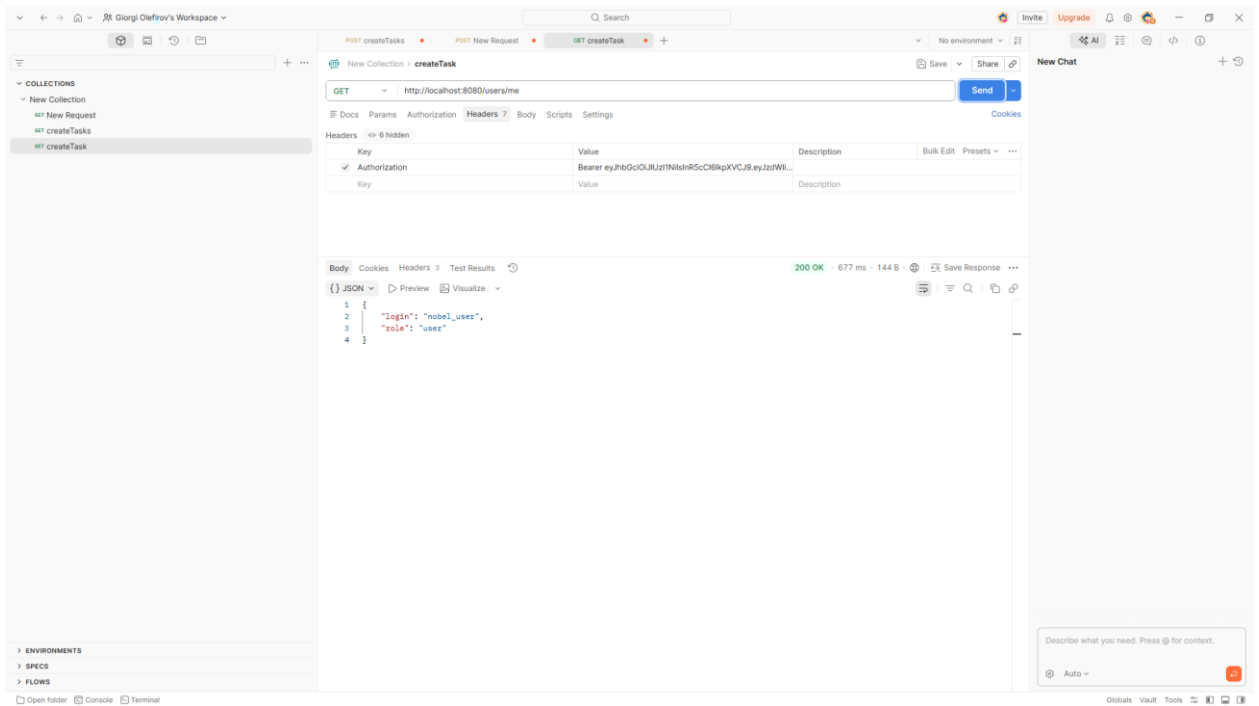


Рисунок 20 – Профиль пользователя

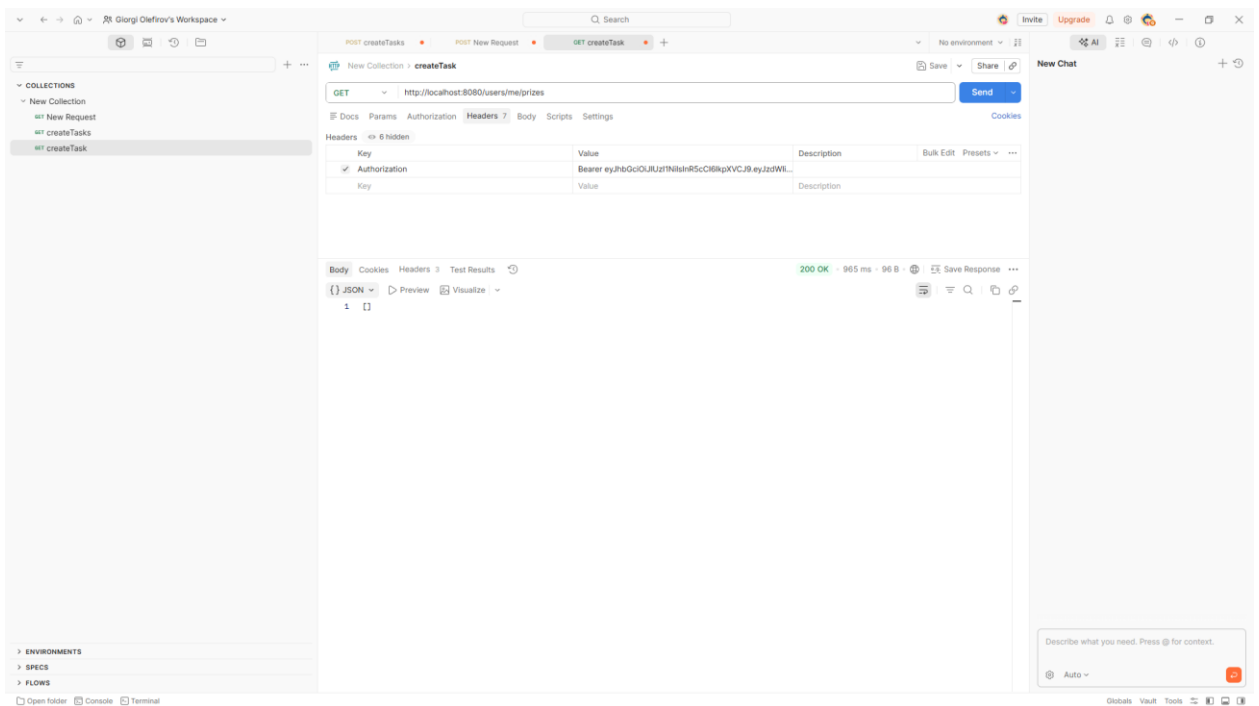


Рисунок 21 – Получение избранных премии

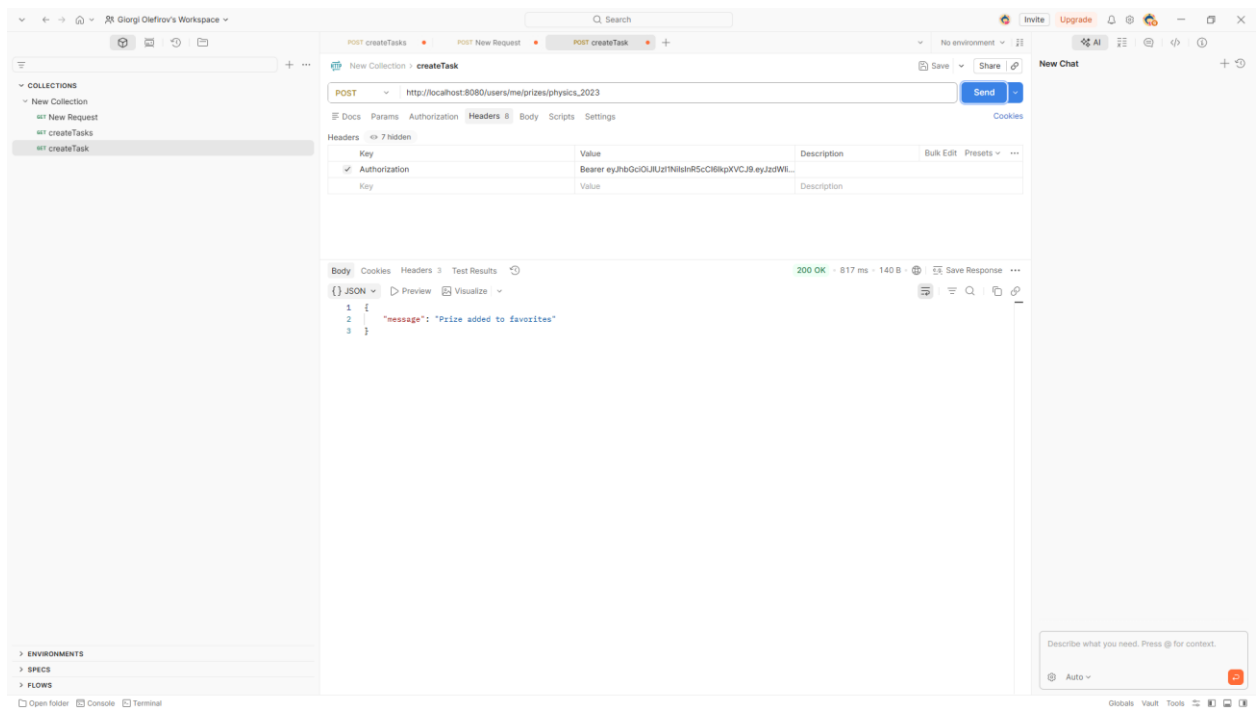


Рисунок 22 – Добавление премии в избранное

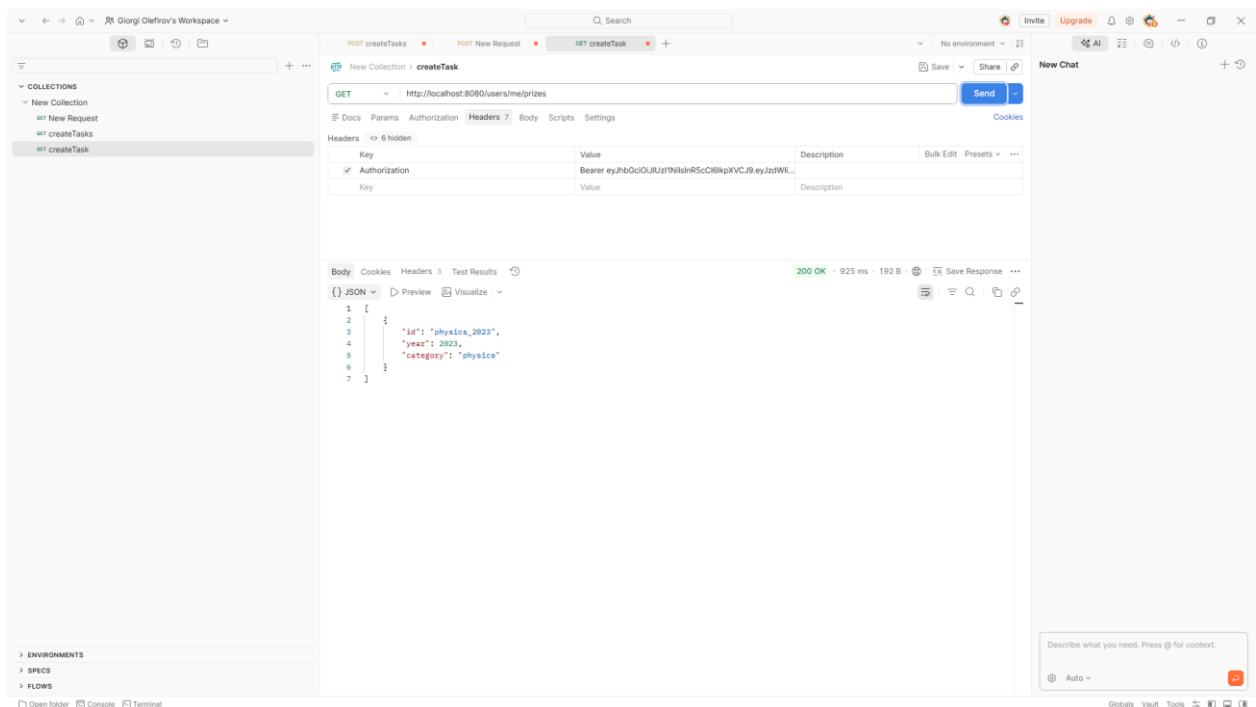


Рисунок 23 – Проверка избранное после добавления

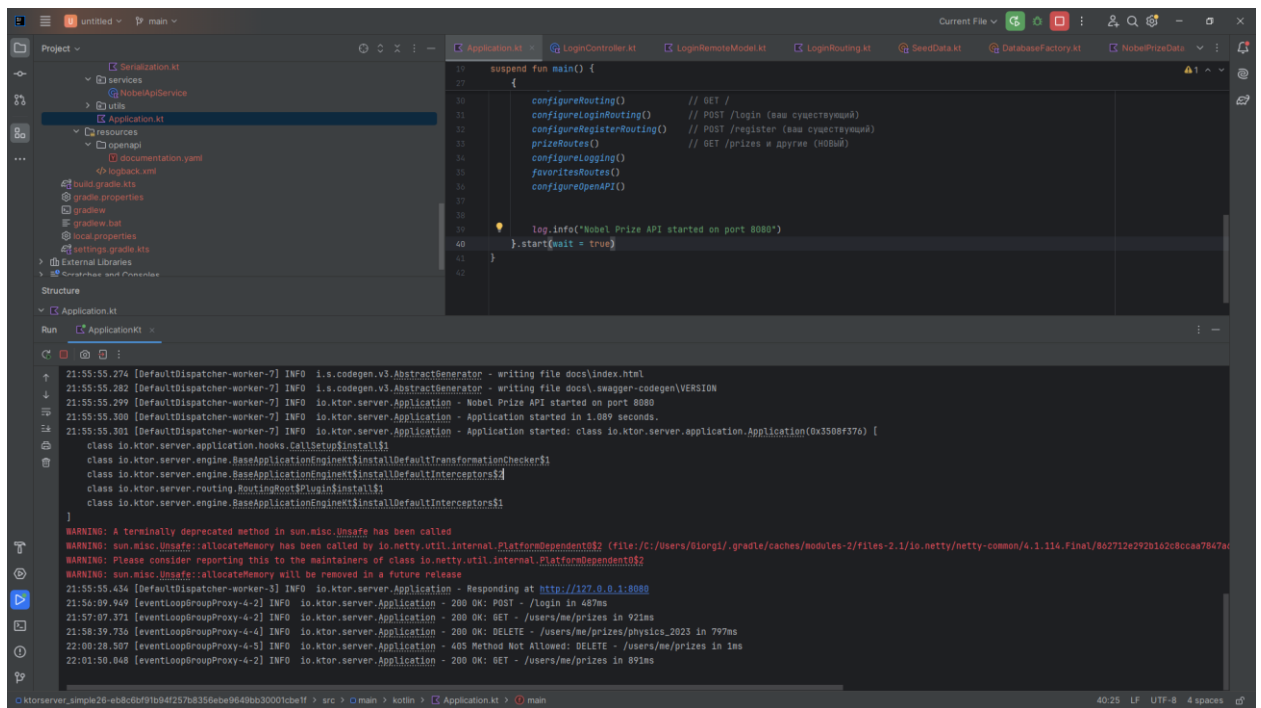


Рисунок 25– Логи

Ссылка на гит:

<https://github.com/oggiorgi/MIREA/tree/main/6sem/kotlin/6module/app/5code>

Задание 6: Доработанное клиентское приложение “Nobel Prize API”

Доработать клиентское приложение из Задания 2 на работу с собственным API сервером из Задания 5. Весь функционал приложения остается таким же.

Моя реализация:

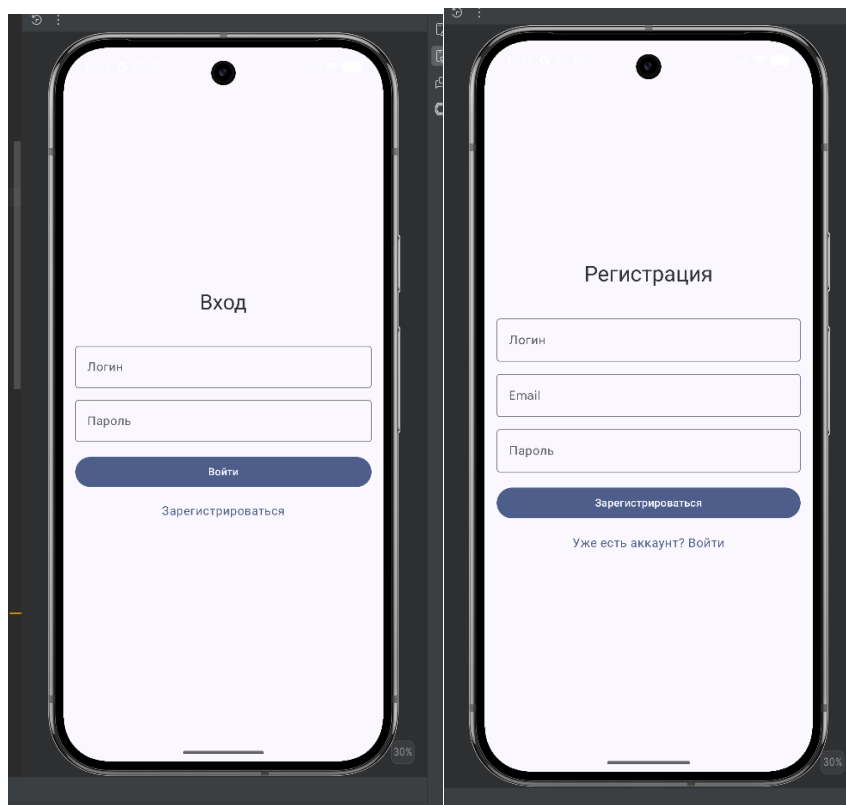


Рисунок 26 – Вход и регистрация

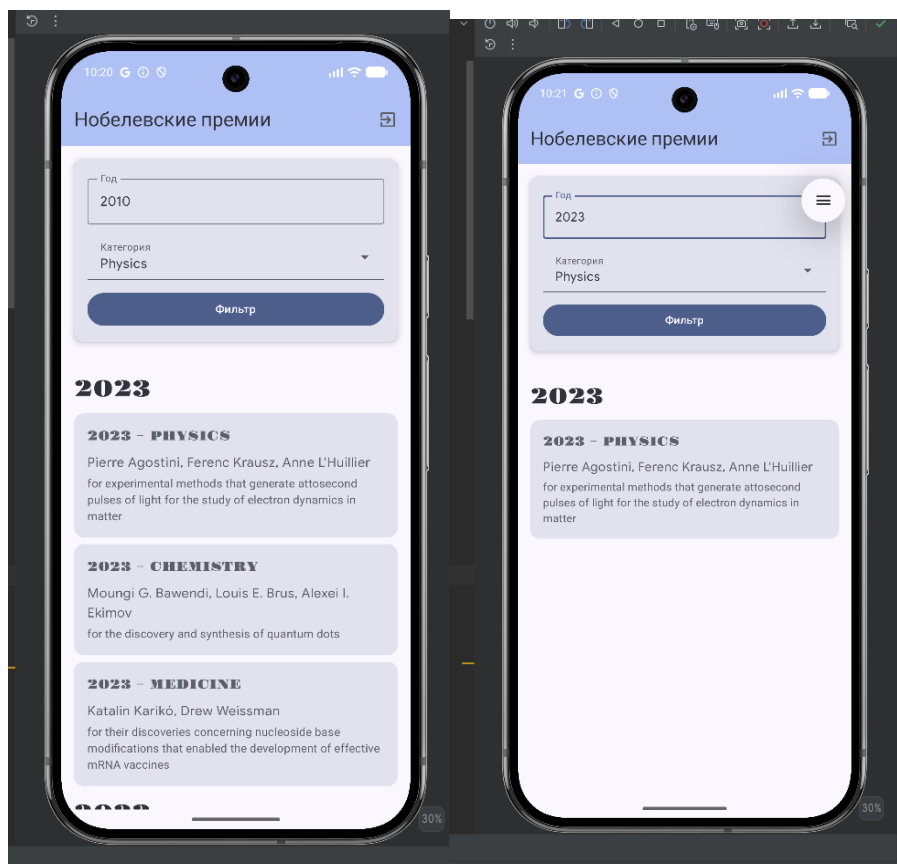


Рисунок 27 - Экран списка лауреатов

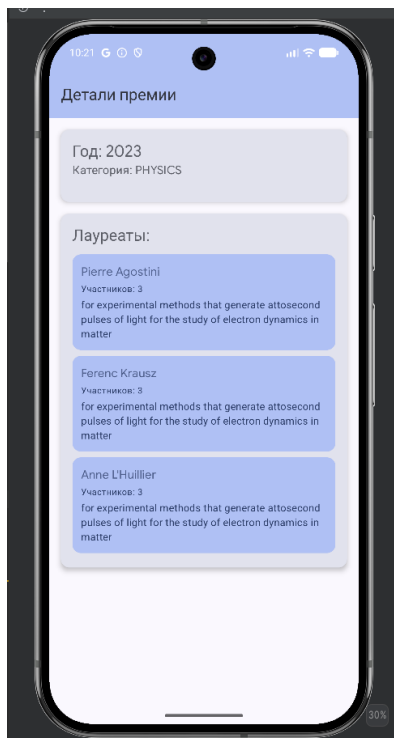


Рисунок 28 - Экран детализации лауреатов

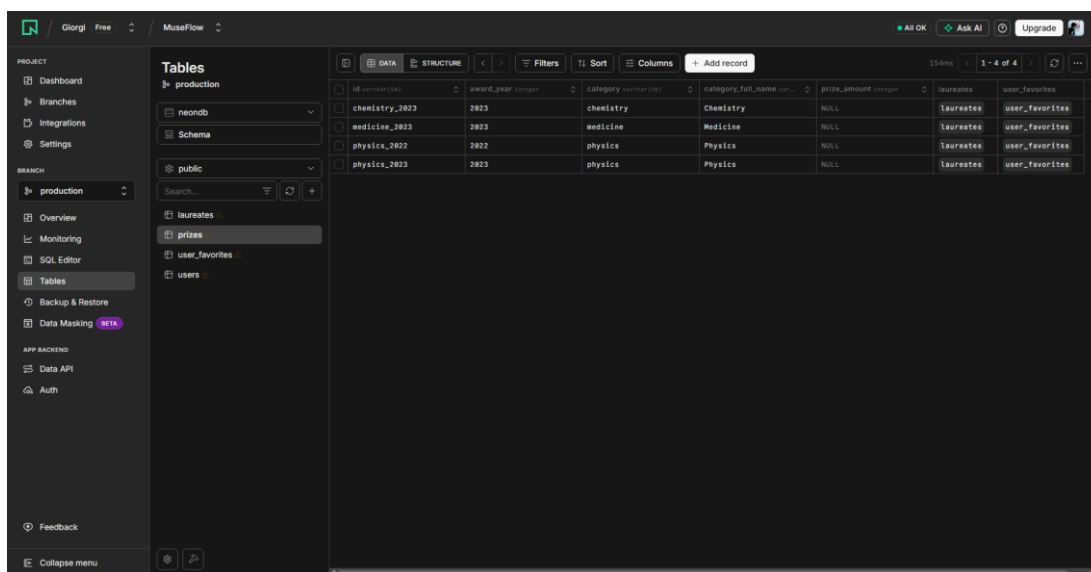


Рисунок 29 – Подключенная бд в Neon.tech

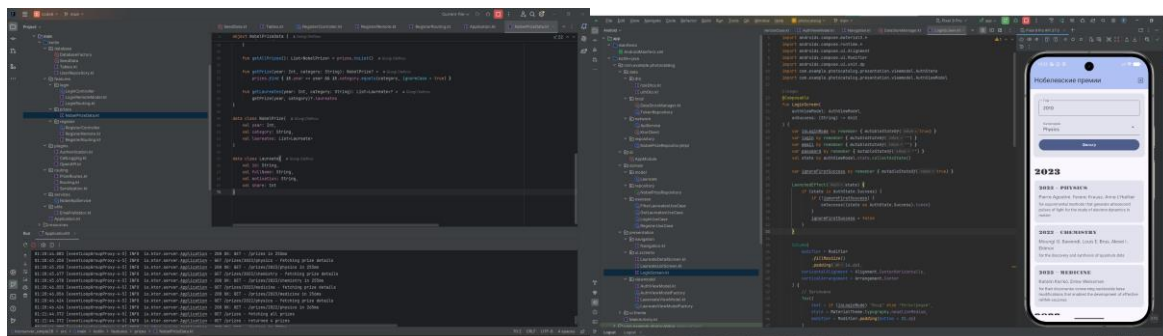


Рисунок 30 – Логи

Ссылка на гит:

<https://github.com/oggiorgi/MIREA/tree/main/6sem/kotlin/6module/app/6code>

Задание 7: BLE приложение “Монитор сердечного ритма”

Модифицируйте представленный пример:

1. Подключается к устройствам с **Heart Rate Service** (UUID 0x180D);
2. Читает и отображает в реальном времени **частоту сердечных сокращений** (Heart Rate Measurement, UUID 0x2A37) через уведомления;

Обязательные экраны и функции:

1. **Главный экран (сканер)** – такой же как в примере.
2. **Вместо давления должен показать** текущую частоту пульса: «Heart Rate: 72 bpm» (или «—»)

Рекомендуемые UUID (стандартные):

- Service: 0000180d-0000-1000-8000-00805f9b34fb (Heart Rate Service)
- Characteristic: 00002a37-0000-1000-8000-00805f9b34fb (Heart Rate Measurement)
- Descriptor: 00002902-0000-1000-8000-00805f9b34fb (Client Characteristic Configuration)

Моя реализация:

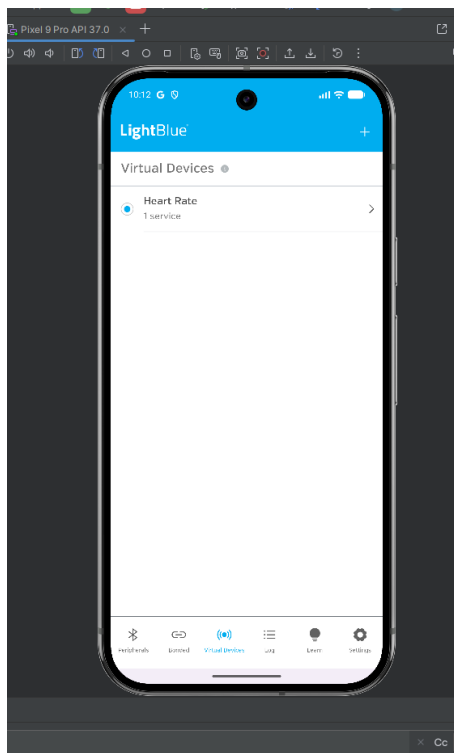


Рисунок 31 — Добавил виртуальное устройство

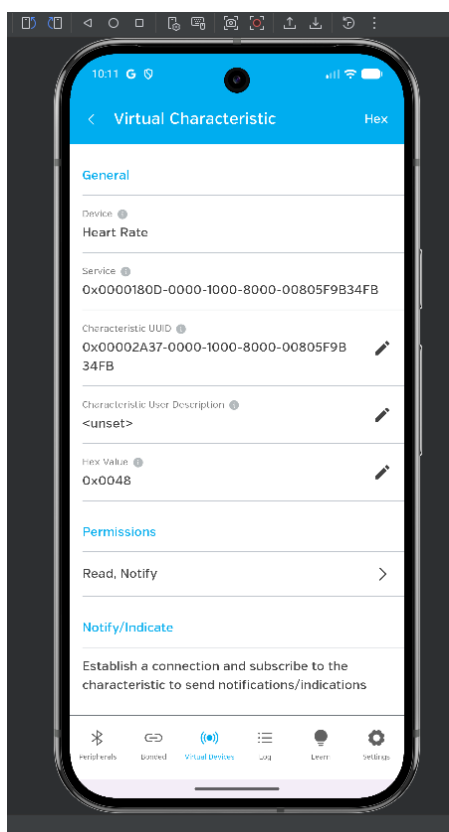


Рисунок 32 – Настраиваем его hex 00 48 и Properties: Read, Notify

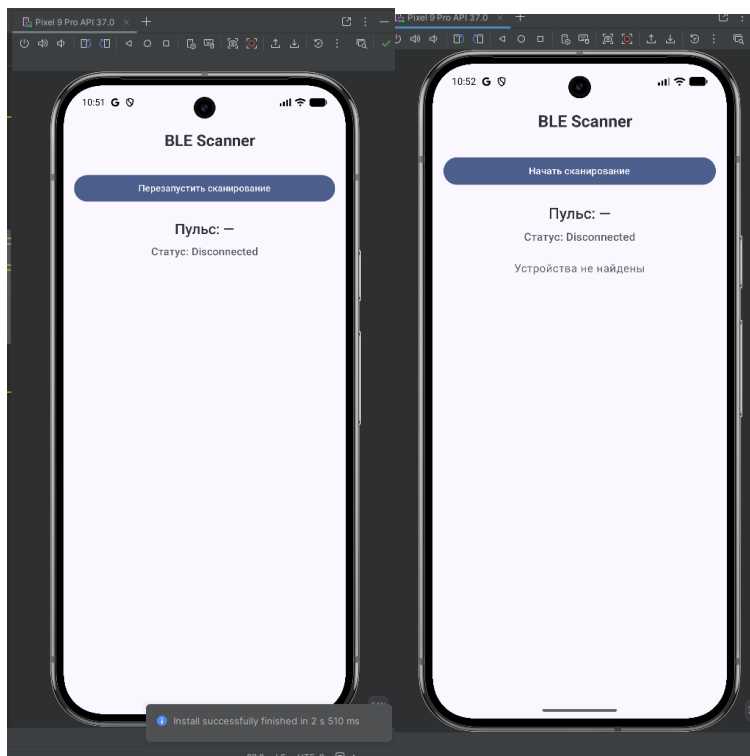


Рисунок 33 – Неработающее приложение 7 на эмуляторе

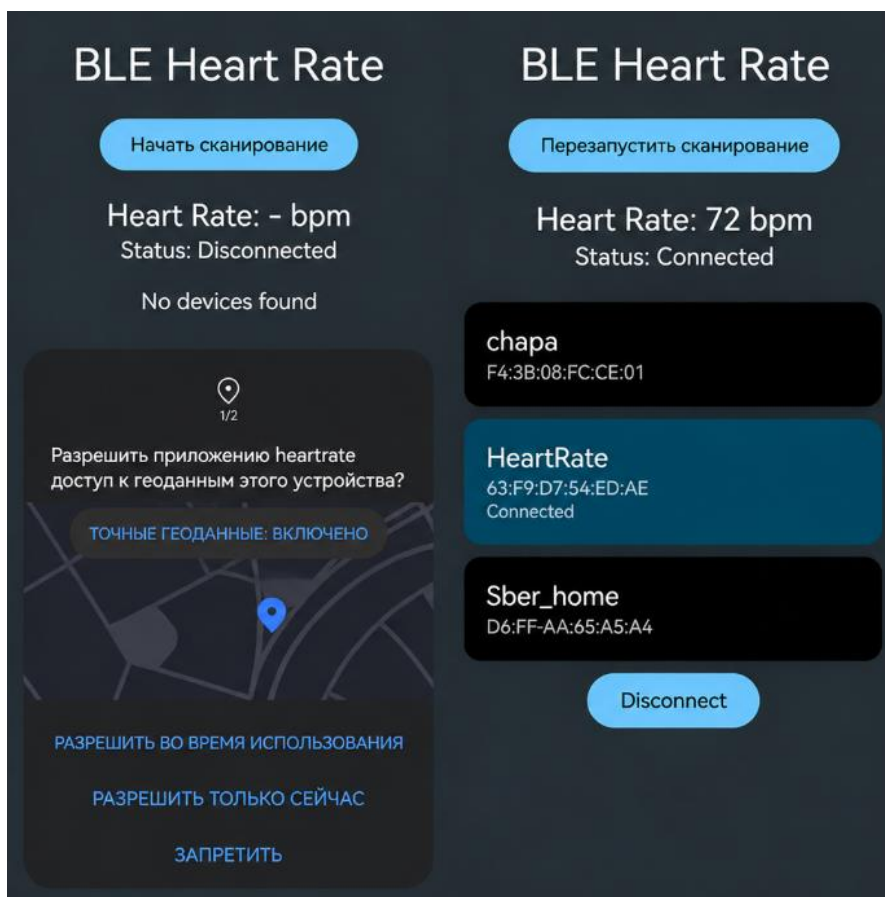


Рисунок 34 – Доработанное приложение 7 на телефоне

Ссылка на гит:

<https://github.com/oggiorgi/MIREA/tree/main/6sem/kotlin/6module/app/7code>

Вывод

В ходе выполнения практических заданий по модулю 6 «Сетевое взаимодействие» были изучены и применены основные подходы к разработке клиент-серверных приложений на Android с использованием современных технологий и архитектурных решений.

Было создано приложение для отображения случайных фотографий с использованием сетевых запросов, загрузкой изображений через Coil и возможностью сохранения фото на устройство. Также разработано приложение со списком Нобелевских лауреатов с фильтрацией, экраном деталей и обработкой состояний интерфейса на основе Clean Architecture.

Реализовано приложение с авторизацией пользователей, сохранением JWT-токена и отображением данных после входа в аккаунт. Дополнительно был создан сервер на Ktor с JWT-авторизацией, защищёнными эндпоинтами и подключением к базе данных PostgreSQL. Добавлена работа с избранным и документация API через SwaggerUI.

Клиентское приложение было адаптировано для работы с собственным сервером, реализованы авторизация, передача токена в запросах и экран избранных элементов.

Также разработано BLE-приложение для поиска устройств с Heart Rate Service, подключения к ним и отображения пульса в реальном времени через

Все приложения демонстрируют работу с сетевыми запросами, базами данных, авторизацией и Bluetooth, а также использование Clean Architecture и Jetpack Compose для создания стабильного и удобного интерфейса.